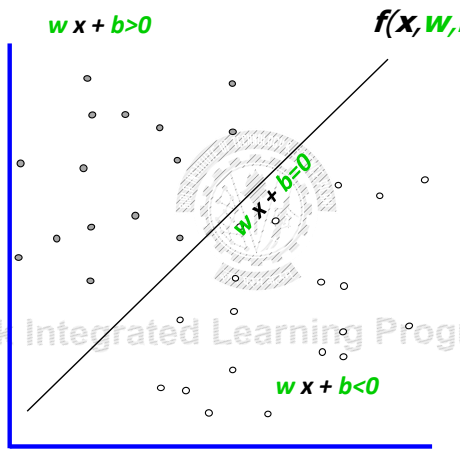
- SVM - I
  - Linear Classifiers
  - Maximum Margin Classification
  - Linear SVM
  - SVM optimization problem
  - Soft Margin SVM

Linear Classifiers

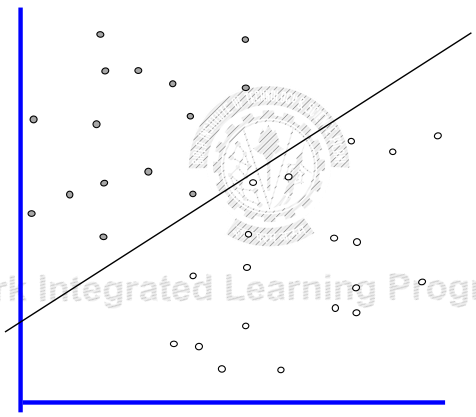
- denotes +1
- denotes -1



How would you classify this data?

Linear Classifiers

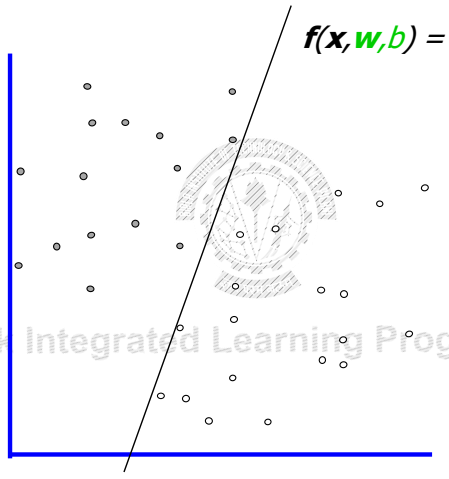
- denotes +1
- denotes -1



How would you classify this data?

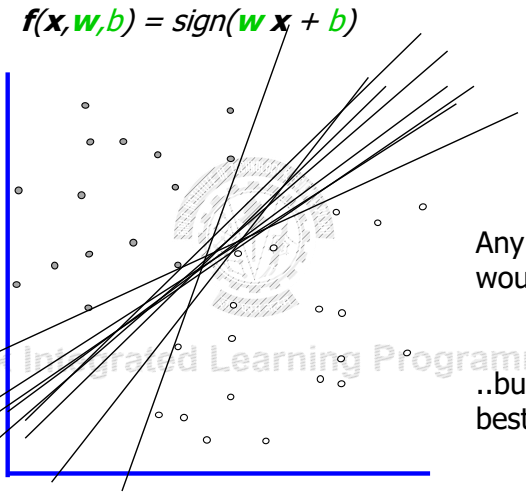
Linear Classifiers

- denotes +1
- denotes -1



How would you classify this data?

Linear Classifiers

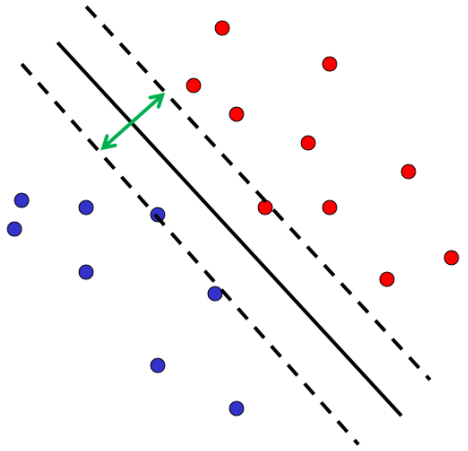


- denotes +1
- denotes -1

Any of these would be fine..

..but which is best?

Linear Classifier

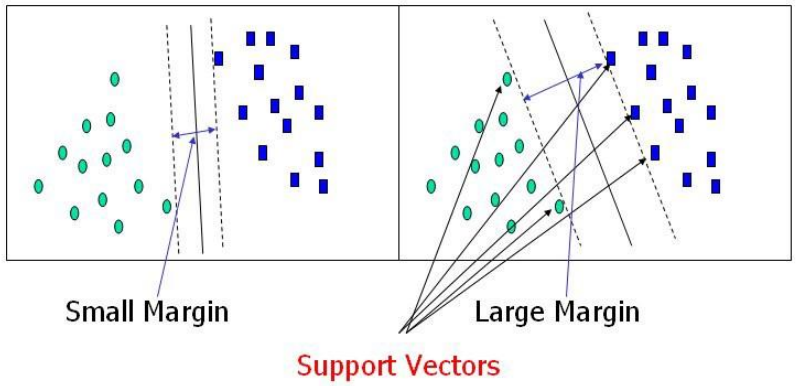


- Discriminative classifier based on *optimal separating line (for 2d case)*
- Maximize the *margin* between the positive and negative training examples

Decision Boundary

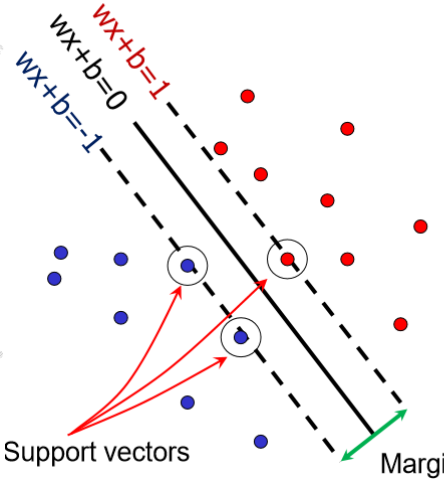
C. Burges, A Tutorial on Support Vector Machines for Pattern Recognition, Data Mining and Knowledge Discovery, 1998

Large margin and support vectors



Support Vector Machines

- Want line that maximizes the margin.

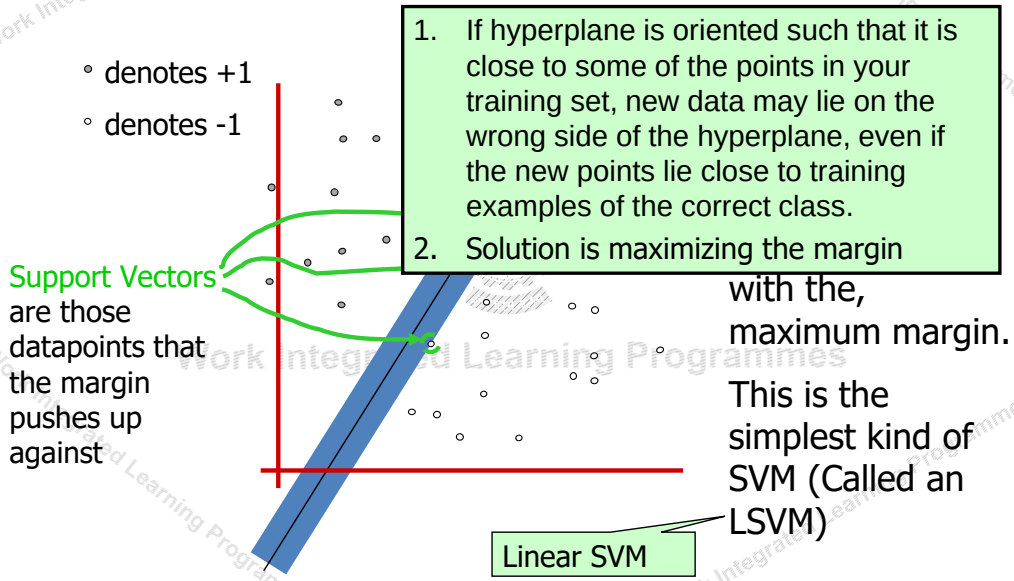


$\mathbf{x}_i$ positive $(y_i = 1)$: $\mathbf{x}_i \cdot \mathbf{w} + b \geq 1$

$\mathbf{x}_i$ negative $(y_i = -1)$: $\mathbf{x}_i \cdot \mathbf{w} + b \leq -1$

For support vectors, $\mathbf{x}_i \cdot \mathbf{w} + b = \pm 1$

Maximum Margin



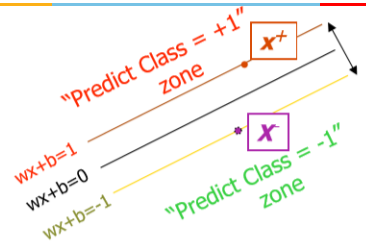
1. If hyperplane is oriented such that it is close to some of the points in your training set, new data may lie on the wrong side of the hyperplane, even if the new points lie close to training examples of the correct class.
2. Solution is maximizing the margin with the, maximum margin.

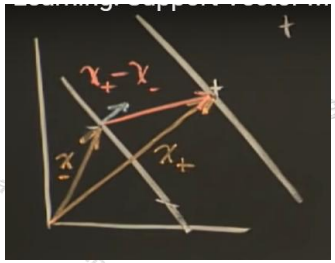
This is the simplest kind of SVM (Called an LSVM)

Support Vectors

- Geometric description of SVM is that the max-margin hyperplane is completely determined by those points that lie nearest to it.
- Points that lie on this margin are the support vectors.
- The points of our data set which if removed, would alter the position of the dividing hyperplane

Linear SVM Mathematically





$\mathbf{w} \cdot \mathbf{x}^+ + b = +1$

$\mathbf{w} \cdot \mathbf{x}^- + b = -1$

Margin width

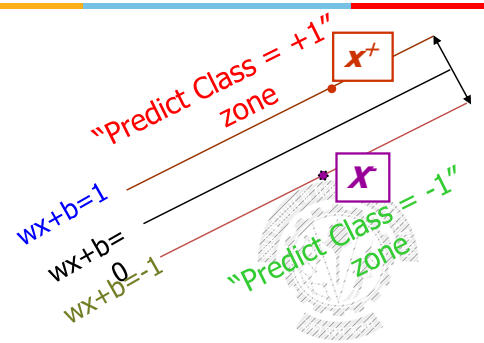
$= \mathbf{x}^+ - \mathbf{x}^- \cdot \frac{\mathbf{w}}{||\mathbf{w}||}$

$= \frac{\mathbf{w} \cdot \mathbf{x}^+ - \mathbf{w} \cdot \mathbf{x}^-}{||\mathbf{w}||}$

$= (1-b) - (-1-b) / ||\mathbf{w}||$

$= \frac{2}{||\mathbf{w}||}$

Linear SVM Mathematically



What we know:

- $\mathbf{w} \cdot \mathbf{x}^+ + b = +1$
- $\mathbf{w} \cdot \mathbf{x}^- + b = -1$

Distance between lines given by solving linear equation:

Maximize margin: $M = \frac{2}{||w||}$

Equivalent to minimize: $\frac{1}{2} ||w||^2$

Solving the Optimization Problem

1. Maximize margin $2/\|\mathbf{w}\|$
2. Correctly classify all training data points:

$$\mathbf{x}_i \text{ positive } (y_i = 1): \quad \mathbf{x}_i \cdot \mathbf{w} + b \geq 1$$
$$\mathbf{x}_i \text{ negative } (y_i = -1): \quad \mathbf{x}_i \cdot \mathbf{w} + b \leq -1$$

Quadratic optimization problem:

Find $\mathbf{w}$ and b such that
$\Phi(\mathbf{w}) = \frac{1}{2}\|\mathbf{w}\|^2$ is minimized;
and for all $\{(\mathbf{x_i}, y_i)\}$: $y_i (\mathbf{w}^T\mathbf{x_i} + b) \geq 1$

$y_i (\mathbf{w}^T\mathbf{x_i} + b) \geq 1$
$+1(\mathbf{w}^T\mathbf{x_i} + b) \geq 1$
$-1(\mathbf{w}^T\mathbf{x_i} + b) \leq 1$
same as $y_i(\mathbf{w}^T\mathbf{x_i} + b) \geq 1$

Solving the Optimization Problem

Find $\mathbf{w}$ and b such that
$\Phi(\mathbf{w}) = \frac{1}{2}\|\mathbf{w}\|^2$ is minimized; Type equation here.
and for all $\{(\mathbf{x_i}, y_i)\}$: $y_i (\mathbf{w}^T\mathbf{x_i} + b) \geq 1$

← Primal

- **Need to optimize a *quadratic* function subject to *linear inequality* constraints.**
- **All constraints in SVM are linear**
- **Quadratic optimization problems are a well-known class of mathematical programming problems, and many (rather intricate) algorithms exist for solving them.**
- **The solution involves constructing a *unconstrained problem* where a *Lagrange multiplier* α_i is associated with every constraint in the primary problem:**

Optimization Problem

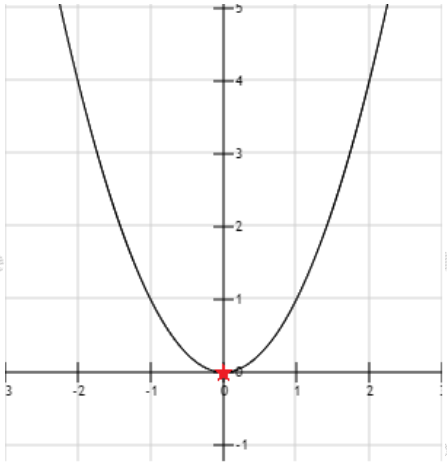
- Optimization problem is typically written:

Minimize f(x)
subject to
$g_i(x) = 0, \quad i=1,\ldots,p$
$h_i(x) <= 0, \quad i=1,\ldots,m$

- f(x) is called the objective function
- By changing x (the optimization variable) we wish to find a value $x*$ for which f(x) is at its minimum.
- p functions of g_i define equality constraints and
- m functions h_i define inequality constraints.
- The value we find MUST respect these constraints!

Unconstrained Optimization

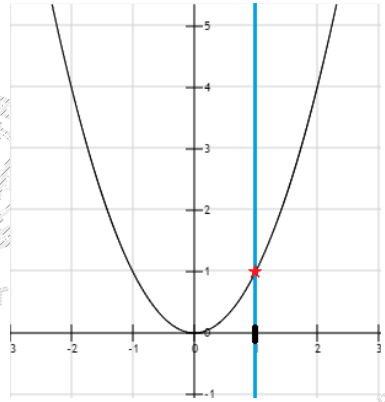
- Minimize x^2



Constrained Optimization -Equality Constraint

Minimize x^2

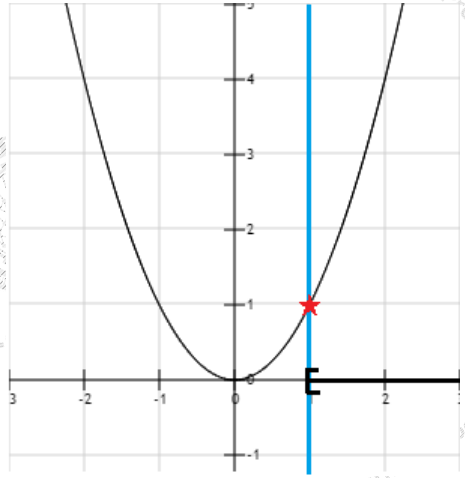
Subject to x = 1



Constrained Optimization -Inequality Constraint

Minimize x^2

Subject to x >= 1



Constrained optimization

- We can also have mix equality and inequality constraints together.
- Only restriction is that if we use contradictory constraints, we can end up with a problem which does not have a feasible set

  Minimize x^2

  Subject to

  x = 1

  x < 0

  Impossible for x to be equal 1 and less than zero at the same

Constrained optimization

- A solution is an assignment of values to variables.
- A feasible solution is an assignment of values to variables such that all the constraints are satisfied.
- The objective function value of a solution is obtained by evaluating the objective function at the given solution.
- An optimal solution (assuming minimization) is one whose objective function value is less than or equal to that of all other feasible solutions.

Lagrange Multipliers

- **How do we find the solution to an optimization problem with constraints?**
- Constrained maximization (minimization) problem is rewritten as a Lagrange function
- *Lagrange function use Lagrange multipliers is a strategy for finding the local maxima and minima of a function subject to constraints*

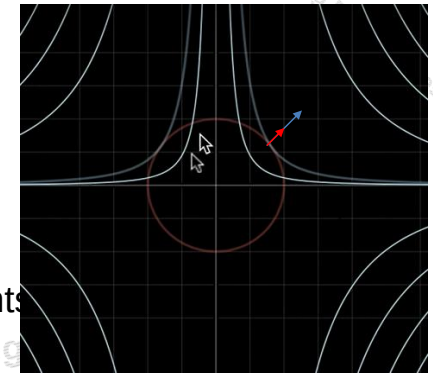
Constrained to Unconstrained Optimization: Lagrange Multiplier

Maximize

$f(x,y) = x^2 y$

Subject to

$g(x, y) : x^2 + y^2 = 1$



- Maximum of f(x,y) under constraint g(x, y) is obtained when their gradients point to same direction (when they are tangent to each other).
- Introduce a Lagrange multiplier λ for the equality constraint
- Mathematically,

$\nabla f(x,y) = \lambda \nabla g(x,y)$

Example:

$$\max_{x,y} xy \text{ subject to } x + y = 6$$

- Introduce a Lagrange multiplier λ for constraint
- Construct the Lagrangian

$$L(x, y) = xy - \lambda(x + y - 6)$$

- Stationary points

$$\frac{\partial L(x, y)}{\partial \lambda} = x + y - 6 = 0$$

$$\left.\begin{aligned} \frac{\partial L(x, y)}{\partial x} &= y - \lambda = 0 \\ \frac{\partial L(x, y)}{\partial y} &= x - \lambda = 0 \end{aligned}\right\} \Rightarrow x = y = \lambda$$

$$\Rightarrow x = y = 3$$

x and y values remain same even if you take +λ or -λ for equality constraint

2 x = 6
x = y = 3
λ = 3

Karush–Kuhn–Tucker (KKT) theorem

- KKT approach to nonlinear programming (quadratic) generalizes the method of Lagrange multipliers, which allows only equality constraints.
- KKT allows inequality constraints

Karush-Kuhn-Tucker (KKT) conditions

- Start with

  max f(x) subject to

  $$g_i(x) = 0 \text{ and } h_j(x) \geq 0 \text{ for all } i, j$$

- Make the Lagrangian function

$$\mathcal{L} = f(x) - \sum_i \lambda_i g_i(x) - \sum_j \mu_j h_j(x)$$

- Take gradient and set to 0 – but other conditions also.

KKT conditions

- Make the Lagrangian function

$$\mathcal{L} = f(x) - \sum_i \lambda_i g_i(x) - \sum_j \mu_j h_j(x)$$

- Necessary conditions to have a minimum are

$$\nabla_x \mathcal{L}(x^*, \lambda^*, \mu^*) = 0$$
$$g_i(x^*) = 0 \text{ for all } i$$
$$h_j(x^*) \geq 0 \text{ for all } j$$
$$\mu_j \geq 0 \text{ for all } j$$
$$\mu_j^* h_j(x^*) = 0 \text{ for all } j$$

Solving the Optimization Problem

- **The solution involves constructing a *dual problem* where a *Lagrange multiplier* α_i is associated with every constraint in the primary problem:**

$$L(w, b, \alpha_i) = \frac{1}{2}\|w\|^2 - \Sigma\, \alpha_i [y_i (\mathbf{w}^T \mathbf{x_i} + b) - 1]$$

- **Taking partial derivative with respect to w , $\frac{\partial L}{\partial w} = 0$**
  - $w - \Sigma\, \alpha_i y_i \mathbf{x_i} = 0$
  - $w = \Sigma\, \alpha_i y_i \mathbf{x_i}$
- **Taking partial derivative with respect to b, $\frac{\partial L}{\partial b} = 0$**
  - $-\Sigma\, \alpha_i y_i = 0$
  - $\Sigma\, \alpha_i y_i = 0$

Solving the Optimization Problem

$$L(w, b, \alpha_i) = \frac{1}{2}\|w\|^2 - \Sigma\, \alpha_i [y_i (\mathbf{w} \cdot \mathbf{x_i} + b) - 1]$$

- Expanding above equation:

$$L(w, b, \alpha_i) = \frac{1}{2}\|w\|^2 - \Sigma\, \alpha_i y_i \mathbf{w} \cdot \mathbf{x_i} - \Sigma\, \alpha_i y_i b + \Sigma\, \alpha_i$$

- Substituting $w = \Sigma\, \alpha_i y_i \mathbf{x_i}$ and $\Sigma\, \alpha_i y_i = 0$ in above equation

$$L(w, b, \alpha_i) = \frac{1}{2}\left(\sum_i \alpha_i y_i \mathbf{x_i}\right)\left(\sum_j \alpha_j y_j \mathbf{x_j}\right) - \left(\sum_i \alpha_i y_i \mathbf{x_i}\right)\left(\sum_j \alpha_j y_j \mathbf{x_j}\right) + \Sigma\, \alpha_i$$

$$L(w, b, \alpha_i) = \Sigma\, \alpha_i - \frac{1}{2}\left(\sum_i \alpha_i y_i \mathbf{x_i}\right)\left(\sum_j \alpha_j y_j \mathbf{x_j}\right)$$

$$L(w, b, \alpha_i) = \Sigma\, \alpha_i - \frac{1}{2}\left(\sum_i \sum_j \alpha_i \alpha_j y_i y_j \mathbf{x_i} \cdot \mathbf{x_j}\right)$$

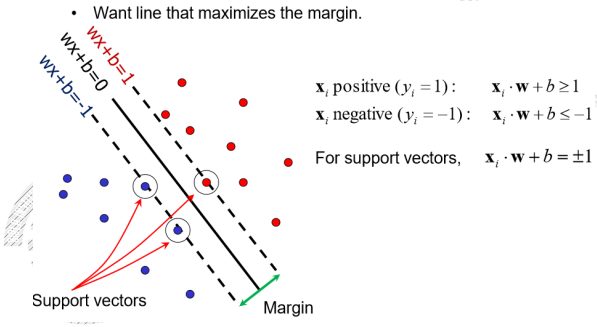
Support Vectors

Using KKT conditions :
$\alpha_i [y_i (\mathbf{w}^T\mathbf{x_i} + b) -1]=0$

For this condition to be satisfied either $\alpha_i =0$ and $y_i (\mathbf{w}^T\mathbf{x_i} + b) -1 > 0$
OR
$y_i (\mathbf{w}^T\mathbf{x_i} + b) -1=0$ and $\alpha_i > 0$

For support vectors:
$y_i (\mathbf{w}^T\mathbf{x_i} + b) -1=0$

For all points other than support vectors:
$\alpha_i =0$



$\mathbf{x}_i$ positive $(y_i = 1)$: $\mathbf{x}_i \cdot \mathbf{w} + b \geq 1$
$\mathbf{x}_i$ negative $(y_i = -1)$: $\mathbf{x}_i \cdot \mathbf{w} + b \leq -1$
For support vectors, $\mathbf{x}_i \cdot \mathbf{w} + b = \pm 1$

$$L(w, b, \alpha_i) = \frac{1}{2}||\mathbf{w}||^2 - \Sigma \alpha_i [y_i (\mathbf{w}^T\mathbf{x_i} + b) -1]$$

Solving the Optimization Problem

- Solution: $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$

Learned weight

Support vector

Solving the Optimization Problem

- Solution: $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$

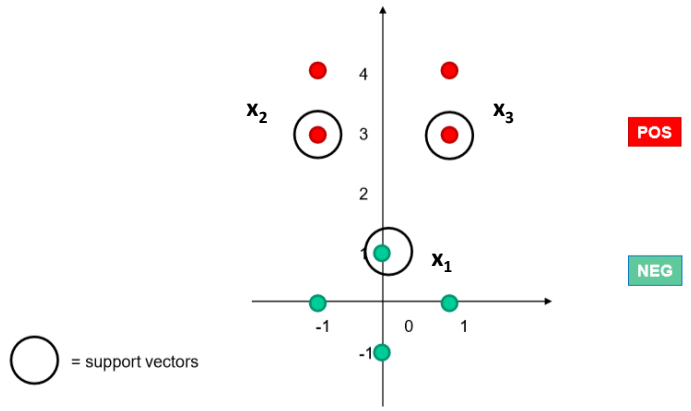
  $b = y_i - \mathbf{w} \cdot \mathbf{x}_i$ (for any support vector)
- Classification function:

$$f(x) = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b)$$
$$= \text{sign}\left(\sum_i \alpha_i y_i \mathbf{x}_i \cdot \mathbf{x} + b\right)$$

If f(x) < 0, classify as negative, otherwise classify as positive.

- Notice that it relies on an *inner product* between the test point ***x*** and the support vectors $\boldsymbol{x}_i$
- (Solving the optimization problem also involves computing the inner products $\boldsymbol{x}_i \cdot \boldsymbol{x}_j$ between all pairs of training points)

Example



Example adapted from Dan Ventura

Solving for α

- We know that for the support vectors, f(x) = 1 or -1 exactly
- Add a 1 in the feature representation for the bias
- The support vectors have coordinates and labels:
  - x1 = [0 1 1], y1 = -1
  - x2 = [-1 3 1], y2 = +1
  - x3 = [1 3 1], y3 = +1
- Thus we can form the following system of linear equations:

Solving for α

- System of linear equations:

α1 y1 dot(x1, x1) + α2 y2 dot(x1, x2) + α3 y3 dot(x1, x3) = y1
α1 y1 dot(x2, x1) + α2 y2 dot(x2, x2) + α3 y3 dot(x2, x3) = y2
α1 y1 dot(x3, x1) + α2 y2 dot(x3, x2) + α3 y3 dot(x3, x3) = y3

-2 * α1 + 4 * α2 + 4 * α3 = -1
-4 * α1 + 11 * α2 + 9 * α3 = +1
-4 * α1 + 9 * α2 + 11 * α3 = +1

$\alpha_i [-1 (\mathbf{w} \cdot \mathbf{x_i} + b)] = -1$
$\alpha_i [+1 (\mathbf{w} \cdot \mathbf{x_i} + b)] = 1$

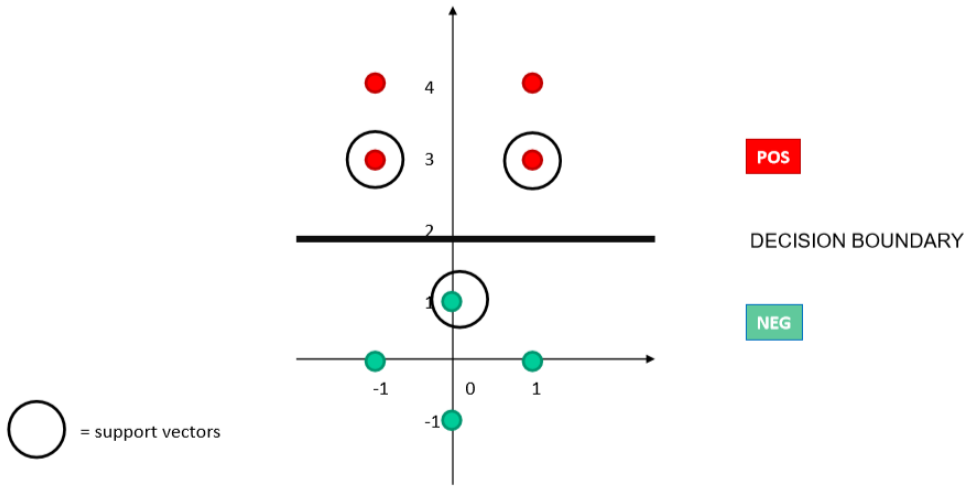
- Solution: α1 = 3.5, α2 = 0.75, α3 = 0.75

Solving for w and b

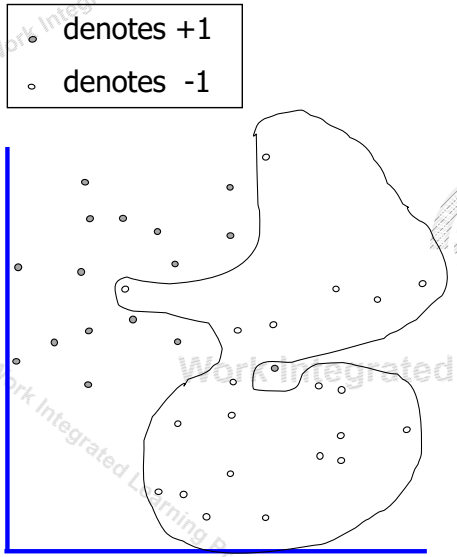
We know $w = \alpha_1 y_1 x_1 + \ldots + \alpha_N y_N x_N$ where N = # SVs
Thus w = -3.5 * [0 1 1] + 0.75 [-1 3 1] + 0.75 [1 3 1] = [0 1 -2]
Separating out weights and bias, we have: w = [0 1] and b = -2

For SVMs, we used this eq for a line: ax + cy + b = 0 where w = [a c]
Thus ax + b = -cy ➔ y = (-a/c) x + (-b/c)
Thus y-intercept is -(-2)/1 = 2
The decision boundary is perpendicular to w and it has slope -0/1 = 0

Decision boundary



Dataset with noise



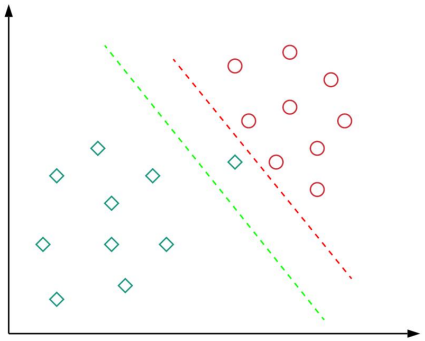
- **Hard Margin: So far we require all data points be classified correctly**

  - No training error

- **What if the training set is noisy?**

- Motivation : Softmargin

- Almost all real-world applications have data that is linearly inseparable.
- When data *is* linearly separable, choosing perfect d[illegible] to overfitting



Soft Margin Classification

Slack variables ξ_i **can be added to allow misclassification of difficult or noisy examples.**

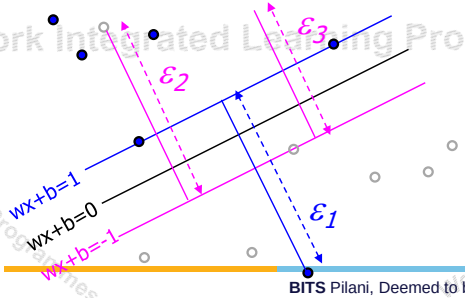
What should our quadratic optimization criterion be?

Minimize

$$\frac{1}{2}\mathbf{w}.\mathbf{w} + C\sum_{k=1}^{R}\varepsilon_k$$

- Soft Margin Classification

- ***Slack variables*** ξ_i **can be added to allow misclassification of difficult or noisy examples.**
- slack variable ξ_i for every data point x_i,
- ξ_i is the distance of x_i from the corresponding class's margin. **It can be regarded as a measure of confidence**
- Points that are far away from the margin on the wrong side would get more penalty

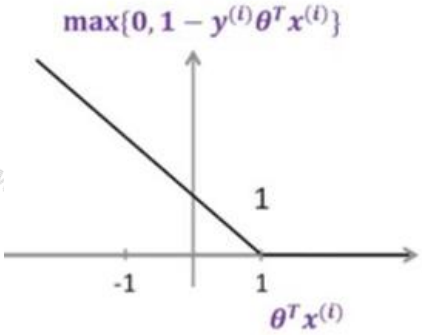


- Slack variable-Hinge loss

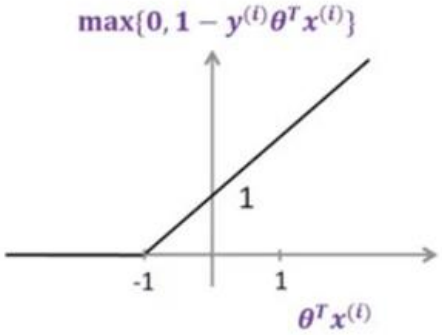
$$\min_{\theta} \underbrace{\frac{1}{2}\|\theta\|^2}_{\text{Margin}} + C \underbrace{\sum_{i=1}^{n} \max\{0, 1 - y^{(i)}\theta^T x^{(i)}\}}_{\text{(hinge loss). Cost/Loss of classifying one data-point}}$$

Regularization parameter

Case where $y^{(i)} = +1$



Case where $y^{(i)} = -1$



Soft Margin

$$\min_{\boldsymbol{w}} \frac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_{i=1}^{N}\xi_i$$

The $\boldsymbol{w}$ that minimizes…

Misclassification cost

N # data samples

Slack variable

Maximize margin

Minimize misclassification

$$\text{subject to} \quad y_i \boldsymbol{w}^T \boldsymbol{x}_i \geq 1 - \xi_i,$$
$$\xi_i \geq 0, \quad \forall i = 1, \ldots, N$$

Hard Margin versus Soft Margin

- **Hard Margin:**

Find **w** and *b* such that
$\Phi(\mathbf{w}) = \frac{1}{2}\mathbf{w}^T\mathbf{w}$ is minimized and for all $\{(\mathbf{x_i}, y_i)\}$
$y_i(\mathbf{w}^T\mathbf{x_i} + b) \geq 1$

- **Soft Margin incorporating slack variables:**

Find **w** and *b* such that
$\Phi(\mathbf{w}) = \frac{1}{2}\mathbf{w}^T\mathbf{w} + C\sum\xi_i$ is minimized and for all $\{(\mathbf{x_i}, y_i)\}$
$y_i(\mathbf{w}^T\mathbf{x_i} + b) \geq 1 - \xi_i$ and $\xi_i \geq 0$ for all i

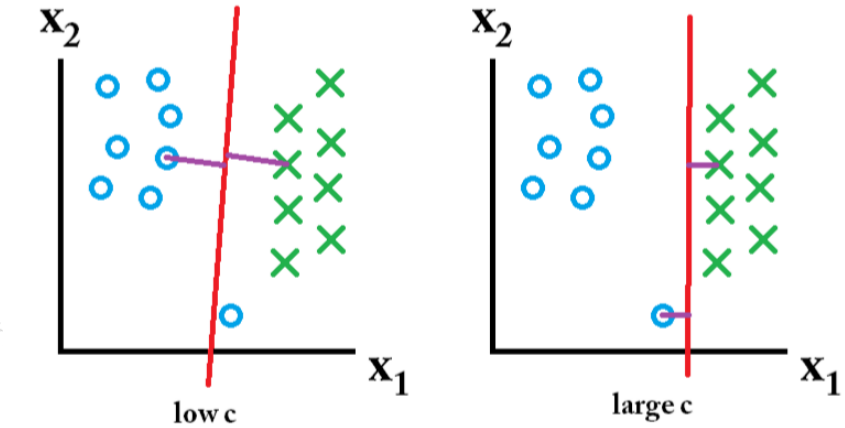
- **Parameter *C* can be viewed as a way to control overfitting.**

Value of C parameter

- C parameter tells the SVM optimization how much you want to avoid misclassifying each training example.
- For large values of C, the optimization will choose a smaller-margin hyperplane if that hyperplane does a better job of getting all the training points classified correctly.
- Conversely, a very small value of C will cause the optimizer to look for a larger-margin separating hyperplane, even if that hyperplane misclassifies more points.

Effect of Margin size v/s misclassification cost

Training set

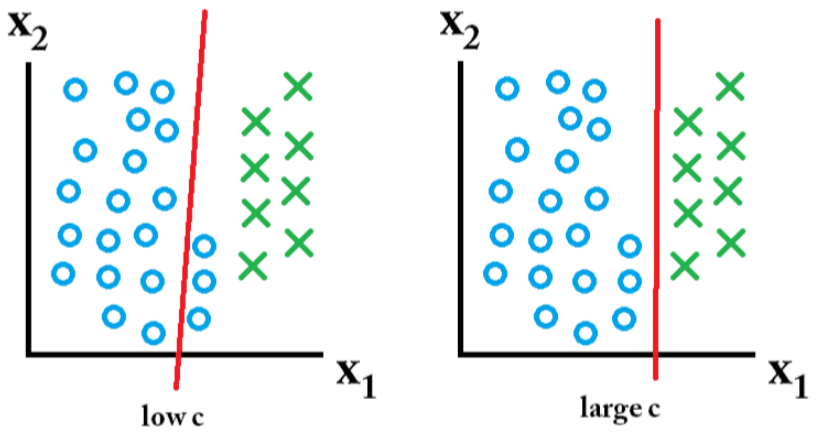


Misclassification ok, want large margin

Misclassification not ok

Effect of Margin size v/s misclassification cost

Including test set A

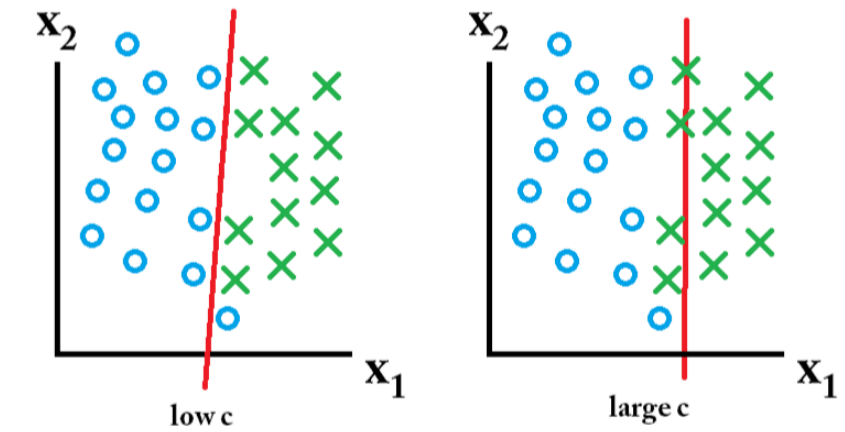


Misclassification ok, want large margin

Misclassification not ok

Effect of Margin size v/s misclassification cost

Including test set B



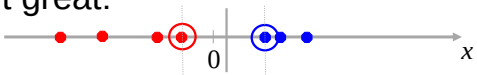
Misclassification ok, want large margin

Misclassification not ok

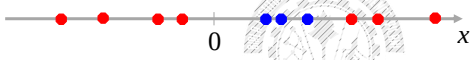
- SVM - II
  - Nonlinear SVM
  - Kernel Trick
  - SVM Kernels
  - Multi-Class Problem
  - SVM Applications

Non-linear SVMs

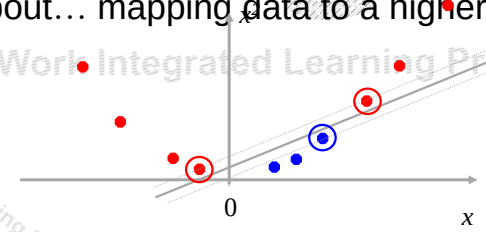
- Datasets that are linearly separable with some noise soft margin work out great:



- But what are we going to do if the dataset is just too hard?

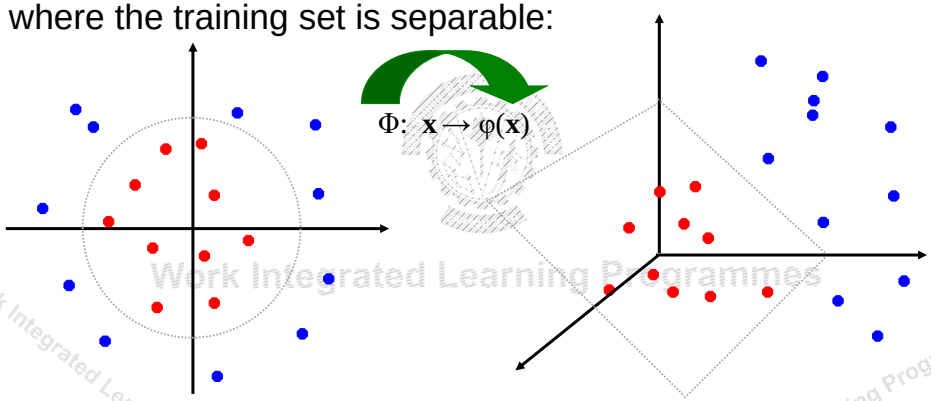


- How about… mapping data to a higher-dimensional space:



Non-linear SVMs: Feature spaces

- General idea: the original input space can always be mapped to some higher-dimensional feature space where the training set is separable:



SVM – Overlapping Class Scenario

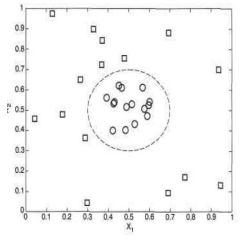
- Data is not separable linearly
- Margin will become inefficient
- Data needs to be transformed from original coordinate space **x** to a new space **Φ(x)**, so that linear decision boundary can be applied
- A non-linear transformation function is needed, like, ex:

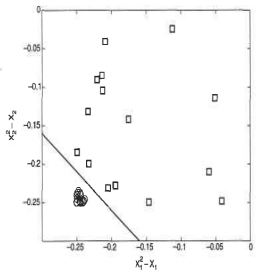
$$\Phi : (x_1, x_2) \longrightarrow (x_1^2, x_2^2, \sqrt{2}x_1, \sqrt{2}x_2, 1)$$

- In the transformed space we can choose w = (w_0, w_1, … , w_4) such that

$$w_4 x_1^2 + w_3 x_2^2 + w_2\sqrt{2}x_1 + w_1\sqrt{2}x_2 + w_0 = 0.$$

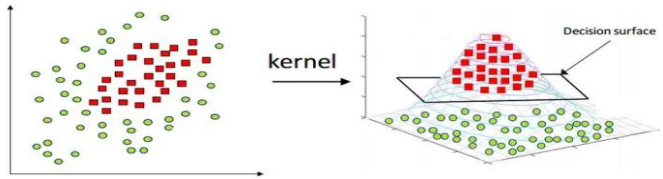
- The linear decision boundary in the transformed space has the following form: w. **Φ**(x) + b = 0





Overlapping class distribution

- Non-Linear SVMs
- Step-1:
  - Transform the original input data into a higher dimensional space using a nonlinear mapping. Once the data have been transformed into the new higher space, go to step 2.
- Step-2:
  - Search for a linear separating hyperplane in the new space.
- We again end up with a quadratic optimization problem that can be solved using the linear SVM formulation.
- The maximal marginal hyperplane found in the new space corresponds to a nonlinear separating hypersurface in the original space.



Overlapping class distribution

Learning a Nonlinear SVM Model

- For non-overlapping classes, the learning task of SVM is defined as the constrained optimization problem

$$\min_{\mathbf{w}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{d}\xi_i \quad \text{Subject to: } y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i,\ \xi_i \geq 0, \forall i = 1, \ldots, d$$

- For non-linear SVM, the learning task can be formalized as:

$$\min_{\mathbf{w}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{d}\xi_i \quad \text{Subject to: } y_i(\mathbf{w}\cdot\phi(\mathbf{x}_i) + b) \geq 1 - \xi_i,\ \xi_i \geq 0, \forall i = 1, \ldots, d$$

- Only difference with Linear SVM is the transformed attributes $\phi(\mathbf{x}_i)$

Overlapping class distribution

Solution with dual Lagrange

- In linear SVM, using lagrangian multipliers, the dual optimization problem

$$\max_{\lambda_i} \sum_{i=1}^{n}\lambda_i - \frac{1}{2}\left(\sum_{i,j}\lambda_i\lambda_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j\right)$$

In the linear SVM, w and b can be found using

$$\mathbf{w} = \sum \lambda_i y_i \mathbf{x}_i$$

$$\lambda_i[y_i(\mathbf{w}\cdot\mathbf{x}_i + b) - 1] = 0$$

- In the nonlinear SVM, the dual Lagrangian is:

$$\max_{\lambda_i} \sum_{i=1}^{n}\lambda_i - \frac{1}{2}\left(\sum_{i,j}\lambda_i\lambda_j y_i y_j \phi(\mathbf{x}_i)\cdot\phi(\mathbf{x}_j)\right) \quad \text{Subject to: } \sum_{i=1}^{n}\lambda_i y_i = 0, \lambda_i \geq 0$$

- Once the Lagrange Multipliers are found using quadratic programming techniques, the parameters w and b can be derived using:

$$\mathrm{w} = \sum_i \lambda_i y_i \phi(\mathbf{x}_i) \qquad \lambda_i\left\{y_i\left(\sum_j \lambda_j y_j \phi(\mathbf{x}_j)\cdot\phi(\mathbf{x}_i) + b\right) - 1\right\} = 0$$

Overlapping class distribution

Solution with dual Lagrange

- In the case of linear SVM, a test instance **z** can be classified using:

$$f(x) = sign(\mathbf{w}\cdot\mathbf{x} + b) = sign\left(\sum \lambda_i y_i \mathbf{x}_i \cdot \mathbf{z} + b\right)$$

Support Vectors | Test Tuple

- In the transformed space, a test instance **z** can be classified as:

$$f(\mathrm{z}) = sign(\mathbf{w}\cdot\phi(\mathbf{z}) + b) = sign\left(\sum_{i=1}^{n}\lambda_i y_i \phi(\mathbf{x}_i)\cdot\phi(\mathbf{z}) + b\right)$$

- Only missing piece of the puzzle is transforming **x** to ϕ(**x**) – called 'Kernel Trick'

SVM Kernels

- Kernel functions only calculate the relationship between every pair of points as if they are in high dimension; they actually do not do the transformation
- Trick: Calculating the higher dimensional relationships without actually transforming the data to higher dimension
- Kernel trick avoids the math that is required to transform data into higher dimensions thus reduces the amount of computations.
- Different SVM algorithms use different types of kernel functions. Example *linear, nonlinear, polynomial, and sigmoid etc.*
- A *kernel function* is some function that corresponds to an inner product in some expanded feature space

$$K(x_i, x_j) = \varphi(x_i)^T\ \varphi\ (x_j)$$

Kernel Trick (SVM)

- E.g. remember the hypothesis function of the original simplified SVM:

$$h_\theta(x) = \theta^T x = \theta_0 + \sum_{i=1}^{n} \alpha_i \, y^{(i)} x^T x^{(i)}$$

  - It involves a dot product between the test data-point x and the support vectors $x^{(i)T}$

- Instead of explicitly mapping the data to a higher dimensional space, we can just use a kernel function, and the hypothesis function would have the same form:

$$h_\theta(x) = \theta^T x = \theta_0 + \sum_{i=1}^{n} \alpha_i \, y^{(i)} \underbrace{k(x^T x^{(i)})}_{z^T z^{(i)}}$$

Because since k is a kernel function, we know that $k(x, x^{(i)}) = \underbrace{\Phi(x)^T}_{z^T} \underbrace{\Phi(x^{(i)})}_{z^{(i)}}$

So we can use the dot product between the higher dimensional vectors, without explicitly knowing them (i.e. a trick).

- Examples of Kernel Functions

- Linear: $K(\mathbf{x_i},\mathbf{x_j}) = \mathbf{x_i}^T \mathbf{x_j}$

- Polynomial of power p: $K(\mathbf{x_i},\mathbf{x_j}) = (1 + \mathbf{x_i}^T \mathbf{x_j})^p$

- Gaussian (radial-basis function network):

$$K(\mathbf{x_i}, \mathbf{x_j}) = \exp\left(-\frac{\|\mathbf{x_i} - \mathbf{x_j}\|^2}{2\sigma^2}\right)$$

- Sigmoid: $K(\mathbf{x_i},\mathbf{x_j}) = \tanh(\beta_0 \mathbf{x_i}^T \mathbf{x_j} + \beta_1)$

- Kernel trick example – Polynomial kernel

$$\phi(\mathbf{a})^T \cdot \phi(\mathbf{b}) = \begin{pmatrix} a_1^2 \\ \sqrt{2}\, a_1 a_2 \\ a_2^2 \end{pmatrix}^T \cdot \begin{pmatrix} b_1^2 \\ \sqrt{2}\, b_1 b_2 \\ b_2^2 \end{pmatrix} = a_1^2 b_1^2 + 2 a_1 b_1 a_2 b_2 + a_2^2 b_2^2$$

$$= (a_1 b_1 + a_2 b_2)^2 = \left(\begin{pmatrix} a_1 \\ a_2 \end{pmatrix}^T \cdot \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \right)^2 = (\mathbf{a}^T \cdot \mathbf{b})^2$$

- Kernel example

3 dimensional to be converted into 9 dimensional feature space

$$\mathbf{x} = (x_1, x_2, x_3)^T$$
$$\mathbf{y} = (y_1, y_2, y_3)^T$$

$$\phi(\mathbf{x}) = (x_1^2, x_1x_2, x_1x_3, x_2x_1, x_2^2, x_2x_3, x_3x_1, x_3x_2, x_3^2)^T$$
$$\phi(\mathbf{y}) = (y_1^2, y_1y_2, y_1y_3, y_2y_1, y_2^2, y_2y_3, y_3y_1, y_3y_2, y_3^2)^T$$

$$\phi(\mathbf{x})^T \phi(\mathbf{y}) = \sum_{i,j=1}^{3} x_i x_j y_i y_j$$

Kernel trick:

$$k(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T \mathbf{y})^2$$
$$= (x_1 y_1 + x_2 y_2 + x_3 y_3)^2$$
$$= \sum_{i,j=1}^{3} x_i x_j y_i y_j$$

Kernel: Example

Example:

Let $x^{(i)} = \begin{bmatrix} x_1^{(i)} \\ x_2^{(i)} \end{bmatrix}$, $x^{(j)} = \begin{bmatrix} x_1^{(j)} \\ x_2^{(j)} \end{bmatrix}$, $k(x^{(i)}, x^{(j)}) = \left(1 + x^{(i)T} x^{(j)}\right)^2$

- Is this a kernel function ?
- We need to show that $k(x^{(i)}, x^{(j)}) = \Phi(x^{(i)})^T \Phi(x^{(j)})$

Kernel: Example

Example:

Let $x^{(i)} = \begin{bmatrix} x_1^{(i)} \\ x_2^{(i)} \end{bmatrix}$, $x^{(j)} = \begin{bmatrix} x_1^{(j)} \\ x_2^{(j)} \end{bmatrix}$, $k(x^{(i)}, x^{(j)}) = \left(1 + x^{(i)T} x^{(j)}\right)^2$

$$k(x^{(i)}, x^{(j)}) = 1 + x_1^{(i)2} x_1^{(j)2} + 2x_1^{(i)} x_1^{(j)} x_2^{(i)} x_2^{(j)} + x_2^{(i)2} x_2^{(j)2} + 2x_1^{(i)} x_1^{(j)} + 2x_2^{(i)} x_2^{(j)}$$

$$= \underbrace{\begin{bmatrix} 1 & x_1^{(i)2} & \sqrt{2}x_1^{(i)} x_2^{(i)} & x_2^{(i)2} & \sqrt{2}x_1^{(i)} & \sqrt{2}x_2^{(i)} \end{bmatrix}}_{\Phi\left(x^{(i)}\right)^T} \underbrace{\begin{bmatrix} 1 \\ x_1^{(j)2} \\ \sqrt{2}x_1^{(j)} x_2^{(j)} \\ x_2^{(j)2} \\ \sqrt{2}x_1^{(j)} \\ \sqrt{2}x_2^{(j)} \end{bmatrix}}_{\Phi(x^{(j)})}$$

So, yes, this is a kernel function.

Mercer kernels

- What functions are valid kernels that correspond to feature vectors φ(x)?

- Mercer kernels K
  - K is continuous
  - K is symmetric :

$$K(\vec{x_i}, \vec{x_j}) = \phi(\vec{x_i})^T \phi(\vec{x_j}) \Longrightarrow \phi(\vec{x_j})^T \phi(\vec{x_i}) = K(\vec{x_j}, \vec{x_i})$$

  - K is positive semi-definite, i.e. $x^T K x \geq 0$ for all x
  - Ensures optimization is concave maximization

What Functions are Kernels?

1) We can *construct kernels from scratch*:
   - For any $\varphi : \mathcal{X} \to \mathbb{R}^m$, $k(x, x') = \langle \varphi(x), \varphi(x') \rangle_{\mathbb{R}^m}$ is a kernel.
   - If $d : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is a *distance function*, i.e.
     - $d(x, x') \geq 0$ for all $x, x' \in \mathcal{X}$,
     - $d(x, x') = 0$ only for $x = x'$,
     - $d(x, x') = d(x', x)$ for all $x, x' \in \mathcal{X}$,
     - $d(x, x') \leq d(x, x'') + d(x'', x')$ for all $x, x', x'' \in \mathcal{X}$,

   then $k(x, x') := \exp(-d(x, x'))$ is a kernel.

2) We can *construct kernels from other kernels*:
   - if k is a kernel and $\alpha > 0$, then αk and $k + \alpha$ are kernels.
   - if k_1, k_2 are kernels, then $k_1 + k_2$ and $k_1 \cdot k_2$ are kernels.

- Using a Different Kernel in the Dual Optimization Problem

$f(x) = \Sigma \alpha_i y_i x_i^T x + b$

$$f(x) = \Sigma \alpha_i y_i K(x_i, x_j) + b$$

- For example, using the polynomial kernel with d = 4 (including lower-order terms)→ $(<x_i, x_j> + 1)^4$
- Maximize over α

$$W(\alpha) = \Sigma_i \alpha_i - 1/2 \Sigma_{i,j} \alpha_i \alpha_j y_i y_j <x_i, x_j>$$

- Subject to

$$\alpha_i \geq 0$$

$$\Sigma_i \alpha_i y_i = 0$$

- Hypothesis function
  - $f(x) = \text{sign}(\Sigma_i \alpha_i y_i <x, x_i> + b)$

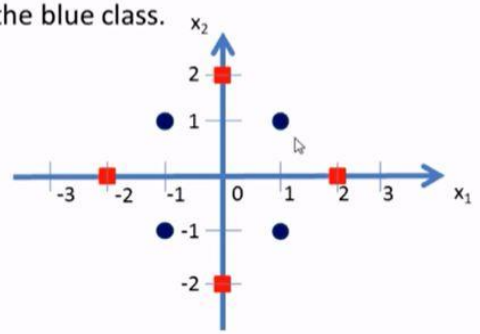
These are kernels!

$(<x_i, x_j> + 1)^4$

So by the kernel trick, we just replace them!

Example-1

- Obviously there is no clear separating hyperplane between the red class and the blue class.



- Blue class vectors are: $\binom{1}{1}, \binom{-1}{1}, \binom{-1}{-1}, \binom{1}{-1}$
- Red class vectors are: $\binom{2}{0}, \binom{0}{2}, \binom{-2}{0}, \binom{0}{-2}$

Example-1

- Here we need to find a non-linear mapping function Φ which can transform these data in to a new feature space where a seperating hyperplane can be found.
- Let us consider the following mapping function.

$$\Phi \binom{x_1}{x_2} = \begin{cases} \binom{6 - x_1 + (x_1 - x_2)^2}{6 - x_2 + (x_1 - x_2)^2} & \text{if } \sqrt{x_1^2 + x_2^2} \geq 2 \\ \binom{x_1}{x_2} & \text{otherwise} \end{cases}$$

Example-1

- Now let us transform the blue and red calss vectors using the non-linear mapping function Φ .

$$\Phi \binom{x_1}{x_2} = \begin{cases} \binom{6 - x_1 + (x_1 - x_2)^2}{6 - x_2 + (x_1 - x_2)^2} & \text{if } \sqrt{x_1^2 + x_2^2} \geq 2 \\ \binom{x_1}{x_2} & \text{otherwise} \end{cases}$$

- Blue class vectors are: $\binom{1}{1}, \binom{-1}{1}, \binom{-1}{-1}, \binom{1}{-1}$ no change since $\sqrt{x_1^2 + x_2^2} < 2$ for all the vectors

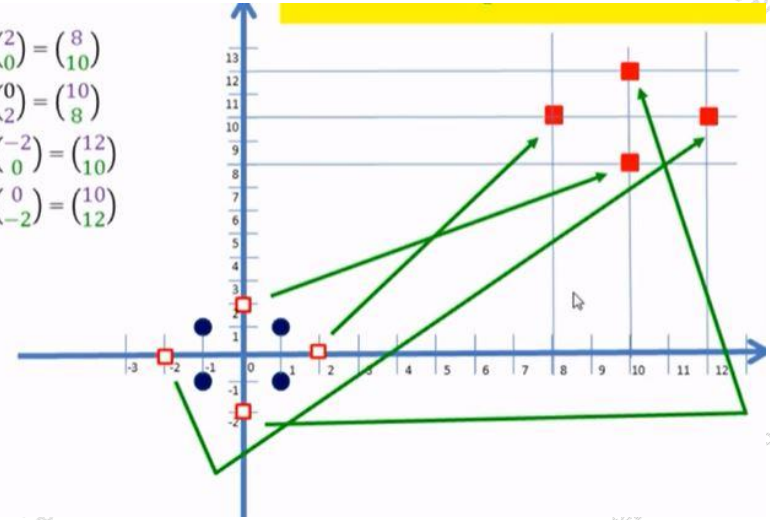
Example-1

- $\Phi\begin{pmatrix}x_1\\x_2\end{pmatrix} = \begin{cases}\begin{pmatrix}6 - x_1 + (x_1 - x_2)^2\\6 - x_2 + (x_1 - x_2)^2\end{pmatrix} & \text{if } \sqrt{x_1^2 + x_2^2} \geq 2\\ \begin{pmatrix}x_1\\x_2\end{pmatrix} & \text{otherwise}\end{cases}$
- Let us take Red class vectors : $\begin{pmatrix}2\\0\end{pmatrix}, \begin{pmatrix}0\\2\end{pmatrix}, \begin{pmatrix}-2\\0\end{pmatrix}, \begin{pmatrix}0\\-2\end{pmatrix}$

- $\Phi\begin{pmatrix}x_1\\x_2\end{pmatrix} = \Phi\begin{pmatrix}2\\0\end{pmatrix} = \begin{pmatrix}6 - 2 + (2 - 0)^2\\6 - 0 + (2 - 0)^2\end{pmatrix} = \begin{pmatrix}8\\10\end{pmatrix}$
- $\Phi\begin{pmatrix}x_1\\x_2\end{pmatrix} = \Phi\begin{pmatrix}0\\2\end{pmatrix} = \begin{pmatrix}6 - 0 + (0 - 2)^2\\6 - 2 + (0 - 2)^2\end{pmatrix} = \begin{pmatrix}10\\8\end{pmatrix}$
- $\Phi\begin{pmatrix}x_1\\x_2\end{pmatrix} = \Phi\begin{pmatrix}-2\\0\end{pmatrix} = \begin{pmatrix}6 + 2 + (-2 - 0)^2\\6 - 0 + (-2 - 0)^2\end{pmatrix} = \begin{pmatrix}12\\10\end{pmatrix}$
- $\Phi\begin{pmatrix}x_1\\x_2\end{pmatrix} = \Phi\begin{pmatrix}0\\-2\end{pmatrix} = \begin{pmatrix}6 - 0 + (0 + 2)^2\\6 + 2 + (0 + 2)^2\end{pmatrix} = \begin{pmatrix}10\\12\end{pmatrix}$

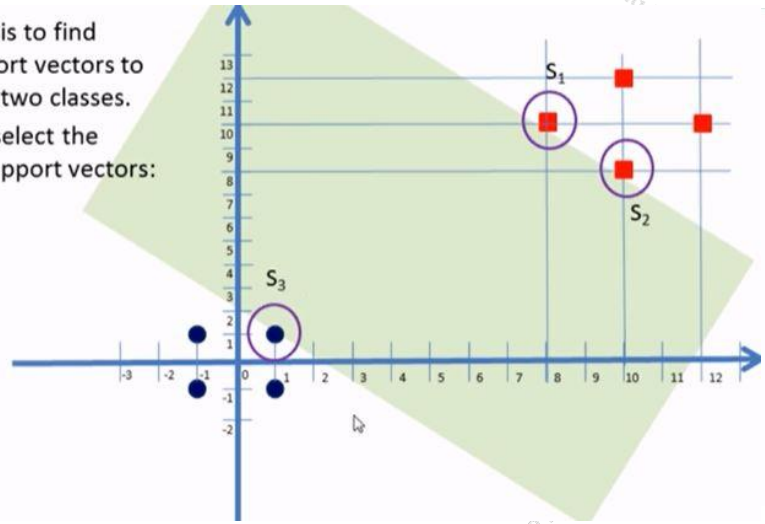
Example-1

- $\Phi\begin{pmatrix}x_1\\x_2\end{pmatrix} = \Phi\begin{pmatrix}2\\0\end{pmatrix} = \begin{pmatrix}8\\10\end{pmatrix}$
- $\Phi\begin{pmatrix}x_1\\x_2\end{pmatrix} = \Phi\begin{pmatrix}0\\2\end{pmatrix} = \begin{pmatrix}10\\8\end{pmatrix}$
- $\Phi\begin{pmatrix}x_1\\x_2\end{pmatrix} = \Phi\begin{pmatrix}-2\\0\end{pmatrix} = \begin{pmatrix}12\\10\end{pmatrix}$
- $\Phi\begin{pmatrix}x_1\\x_2\end{pmatrix} = \Phi\begin{pmatrix}0\\-2\end{pmatrix} = \begin{pmatrix}10\\12\end{pmatrix}$



Example-1

- Now our task is to find suitable support vectors to classify these two classes.
- Here we will select the following 3 support vectors:
- $S_1 = \begin{pmatrix}8\\10\end{pmatrix}$,
- $S_2 = \begin{pmatrix}10\\8\end{pmatrix}$,
- and $S_3 = \begin{pmatrix}1\\1\end{pmatrix}$



Example-1

- Here we will use vectors augmented with a 1 as a bias input, and for clarity we will differentiate these with an over-tilde. That is:

$$S_1 = \begin{pmatrix}8\\10\end{pmatrix} \qquad \widetilde{S_1} = \begin{pmatrix}8\\10\\1\end{pmatrix}$$

$$S_2 = \begin{pmatrix}10\\8\end{pmatrix} \qquad \widetilde{S_2} = \begin{pmatrix}10\\8\\1\end{pmatrix}$$

$$S_3 = \begin{pmatrix}1\\1\end{pmatrix} \qquad \widetilde{S_3} = \begin{pmatrix}1\\1\\1\end{pmatrix}$$

Example-1

$$f(x) = \sum \alpha_i y_i (\mathbf{x}_i{}^\top \mathbf{x}) + b$$

- Now we need to find 3 parameters α_1, α_2, and α_3 based on the following 3 linear equations:

$$\alpha_1 \widetilde{S_1}.\widetilde{S_1} + \alpha_2 \widetilde{S_2}.\widetilde{S_1} + \alpha_3 \widetilde{S_3}.\widetilde{S_1} = +1 \quad (+ve\ class)$$

$$\alpha_1 \widetilde{S_1}.\widetilde{S_2} + \alpha_2 \widetilde{S_2}.\widetilde{S_2} + \alpha_3 \widetilde{S_3}.\widetilde{S_2} = +1 \quad (+ve\ class)$$

$$\alpha_1 \widetilde{S_1}.\widetilde{S_3} + \alpha_2 \widetilde{S_2}.\widetilde{S_3} + \alpha_3 \widetilde{S_3}.\widetilde{S_3} = -1 \quad (-ve\ class)$$

Example-1

- Let's substitute the values for S_1, S_2 and S_3 in the above equations.

$$\widetilde{S_1} = \begin{pmatrix} 8 \\ 10 \\ 1 \end{pmatrix} \quad \widetilde{S_2} = \begin{pmatrix} 10 \\ 8 \\ 1 \end{pmatrix} \quad \widetilde{S_3} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$$

$$\alpha_1 \begin{pmatrix} 8 \\ 10 \\ 1 \end{pmatrix}.\begin{pmatrix} 8 \\ 10 \\ 1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 10 \\ 8 \\ 1 \end{pmatrix}.\begin{pmatrix} 8 \\ 10 \\ 1 \end{pmatrix} + \alpha_3 \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}.\begin{pmatrix} 8 \\ 10 \\ 1 \end{pmatrix} = +1$$

$$\alpha_1 \begin{pmatrix} 8 \\ 10 \\ 1 \end{pmatrix}.\begin{pmatrix} 10 \\ 8 \\ 1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 10 \\ 8 \\ 1 \end{pmatrix}.\begin{pmatrix} 10 \\ 8 \\ 1 \end{pmatrix} + \alpha_3 \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}.\begin{pmatrix} 10 \\ 8 \\ 1 \end{pmatrix} = +1$$

$$\alpha_1 \begin{pmatrix} 8 \\ 10 \\ 1 \end{pmatrix}.\begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 10 \\ 8 \\ 1 \end{pmatrix}.\begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} + \alpha_3 \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}.\begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} = -1$$

Example-1

- After multiplication we get:

$$165\,\alpha_1 + 161\,\alpha_2 + 19\,\alpha_3 = +1$$

$$161\,\alpha_1 + 165\,\alpha_2 + 19\,\alpha_3 = +1$$

$$19\,\alpha_1 + 19\,\alpha_2 + 3\,\alpha_3 = -1$$

- Simplifying the above 3 simultaneous equations we get: $\alpha_1 = \alpha_2 = 0.859$ and $\alpha_3 = -1.4219$.

Example-1

- The hyper plane that discriminates the positive class from the negative class is given by:

$$\mathbf{w} = \Sigma\, \alpha_i y_i \mathbf{x}_i$$

- Substituting the values we get:

$$\tilde{w} = \alpha_1 \begin{pmatrix} 8 \\ 10 \\ 1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 10 \\ 8 \\ 1 \end{pmatrix} + \alpha_3 \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$$

$$\tilde{w} = (0.0859).\begin{pmatrix} 8 \\ 10 \\ 1 \end{pmatrix} + (0.0859).\begin{pmatrix} 10 \\ 8 \\ 1 \end{pmatrix} + (+1.4219).\begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 0.1243 \\ 0.1243 \\ -1.2501 \end{pmatrix}$$

Example-1

For SVMs, we used this eq for a line: ax + cy + b = 0 where w = [a c]
Thus ax + b = -cy ➔ y = (-a/c) x + (-b/c)
Thus y-intercept is (-b/c)
The decision boundary is perpendicular to w and it has slope =(-a/c)

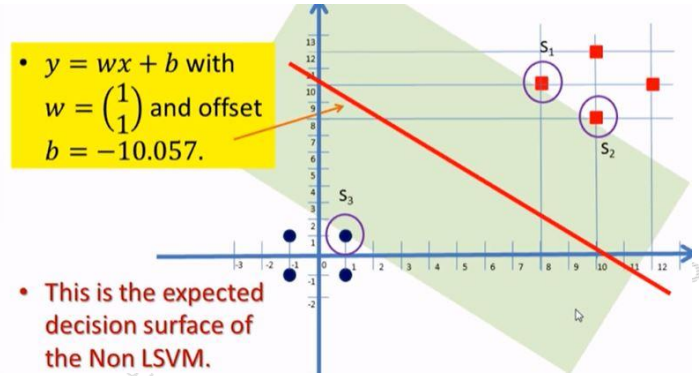
- Our vectors are augmented with a bias.
- Hence we can equate the entry in $\widetilde{w}$ as the hyper plane with an offset b.
- Therefore the separating hyper plane equation

$y = wx + b$ with $w = \begin{pmatrix} 0.1243/0.1243 \\ 0.1243/0.1243 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$

and an offset $b = -\frac{1.2501}{0.1243} = -10.057.$ ← Y intercept

Example-1

- $y = wx + b$ with $w = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$ and offset $b = -10.057$.



- This is the expected decision surface of the Non LSVM.

Non-linear SVM using kernel steps

1. Select a kernel function.
2. Compute pairwise kernel values between labeled examples.
3. Use this “kernel matrix” to solve for SVM support vectors & alpha weights.
4. To classify a new example: compute kernel values between new input and support vectors, apply alpha weights, check sign of output.

SVM versus Logistic Regression

- When viewed from the point of view of regularized empirical loss minimization, SVM and logistic regression appear quite similar:

SVM: $\sum_{i=1}^{n} \left(1 - y_i \left[w_0 + \mathbf{x}_i^T \mathbf{w}_1\right]\right)^+ + \|\mathbf{w}_1\|^2/2$

Logistic: $\sum_{i=1}^{n} \overbrace{-\log \sigma\left(y_i \left[w_0 + \mathbf{x}_i^T \mathbf{w}_1\right]\right)}^{-\log P(y_i|\mathbf{x},\mathbf{w})} + \|\mathbf{w}_1\|^2/2$

where $\sigma(z) = (1 + \exp(-z))^{-1}$ is the logistic function.

(Note that we have transformed the problem maximizing the penalized log-likelihood into minimizing negative penalized log-likelihood.)

SVM versus Logistic Regression

- The difference comes from how we penalize "errors":

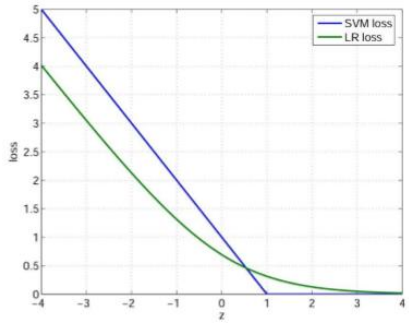
$$\text{Both:} \quad \sum_{i=1}^{n} \text{Loss}\left(\overbrace{y_i\left[w_0 + \mathbf{x}_i^T \mathbf{w}_1\right]}^{z}\right) + \quad \|\mathbf{w}_1\|^2/2$$

- SVM:

$$\text{Loss}(z) = (1-z)^+$$

- Regularized logistic reg:

$$\text{Loss}(z) = \log(1+\exp(-z))$$



SVM versus Logistic Regression

- Logistic regression focuses on maximizing the probability of the data. The farther the data lies from the separating hyperplane (on the correct side), the happier LR is
- An SVM tries to find the separating hyperplane that maximizes the distance of the closest points to the margin (the support vectors). If a point is not a support vector, it doesn't really matter.

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Properties of SVM

- Flexibility in choosing a similarity function
- Sparseness of solution when dealing with large data sets
  - Only support vectors are used to specify the separating hyperplane
  - Therefore SVM also called sparse kernel machine.
- Ability to handle large feature spaces
  - complexity does not depend on the dimensionality of the feature space
- Overfitting can be controlled by soft margin approach
- Nice math property: a simple convex optimization problem which is guaranteed to converge to a single global solution

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

SVMs: Pros and cons

- Pros
  - Kernel-based framework is very powerful, flexible
  - Often a sparse set of support vectors – compact at test time
  - Work very well in practice, even with very small training sample sizes
  - Solution can be formulated as a quadratic program
- Cons
  - Can be tricky to select best kernel function for a problem
  - Computation, memory
    - At training time, must compute kernel values for all example pairs
    - Learning can take a very long time for large-scale problems

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Adapted from Lana Lazebnik

Multi-Class Problem

Instead of just two classes, we now have C classes

- E.g. predict which movie genre a viewer likes best
- Possible answers: action, drama, indie, thriller, etc.

Two approaches:

- One-vs-all
- One-vs-one

Multi-Class Problem

One-vs-all (a.k.a. one-vs-others)

- Train C classifiers
- In each, pos = data from class *i*, neg = data from classes other than *i*
- The class with the most confident prediction wins
- Example:
  - You have 4 classes, train 4 classifiers
  - 1 vs others: score 3.5
  - 2 vs others: score 6.2
  - 3 vs others: score 1.4
  - 4 vs other: score 5.5
  - Final prediction: class 2
- Issues?

Multi-Class Problem

One-vs-one (a.k.a. all-vs-all)

- Train C(C-1)/2 binary classifiers (all pairs of classes)
- They all vote for the label
- Example:
  - You have 4 classes, then train 6 classifiers
  - 1 vs 2, 1 vs 3, 1 vs 4, 2 vs 3, 2 vs 4, 3 vs 4
  - Votes: 1, 1, 4, 2, 4, 4
  - Final prediction is class 4